ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Data Science

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**STUDENT RESULT MANAGEMENT SYSTEM**" is submitted by

S SAILESH	**(24B11DS183)**
Y SATYA ANUNAY	**(24B11DS226)**
A GEETH CHARAN	**(24B11DS004)**
T NITHIN KUMAR	**(24B11DS206)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Data Science for the academic year 2025-2026.

Project Guide
Dr. Karnam.Sreenu,Ph.D,
Assistant Professor,
Department of DS

Head of the Department
Dr. M. Rajababu, Ph.D,
Assistant Professor,
Department of DS

DECLARATION

We hereby declare that the Cornerstone project entitled "**STUDENT RESULT MANAGEMENT SYSTEM**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Data science**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date:

By

S SAILESH (24B11DS183)
Y SATYA ANUNAY (24B11DS226)
A GEETHCHARAN (24B11DS004)
T NITHIN KUMAR (24B11DS206)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr Karnam Sreenu, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. A. Lakshmana Rao, Associate Professor and Mrs. S.Neelima, Assistant Professor**, Department of DS, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. M. Rajababu, Associate Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr.G.Suresh, Registrar,Dr. A.Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S.Rama Sree ,Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of contents

CONTENTS	PAGE NO
ABSTRACT	I
LIST OF FIGURES	II
LIST OF TABLES	III
ACRONYMS	IV
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and contribution of project	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	5
2.3.2 Software Requirements	5
3. TECHNOLOGY DESCRIPTION	6
3.1 Programming Language	6
3.1.1 Introduction to Java	6
3.2 Framework / Libraries	6
3.2.1 Console-Based Interface	6
3.3 Database Technology	7
3.3.1 JDBC (JavaDatabase Connectivity)	7

3.3.2 Oracle Database 7

3.4 Tools & IDEs 8

3.4.1 Visual Studio Code 8

4. DATABASE DESIGN 9

4.1 Database Tables 9

4.2 Table Structure 9

4.3 Keys and Constraints 11

4.4 Table Relationships 12

5. SYSTEM DESIGN 13

5.1 System Architecture 13

5.2 Interface Implementation 13

5.3 Student Management Module 13

5.3.1 Marks And Result Module 13

5.3.2 Subject Management Module 13

5.3.3 Database connectivity module 13

5.3.4 Console interface module 14

5.3.5 User Authentication Module 14

5.3.6 Faculty Assignment Module 14

5.3.7 Branch and Department Module 14

6. SYSTEM IMPLEMENTATION **15**

6.1 Login Screen 15

6.2 Admin Menu and Add Student 15

6.3 Register Course 16

6.4 View Results 16

6.5 Performance Table 17

6.6 Logout 17

6.7 Faculty Menu 18

6.8 Faculty Menu – Enter Marks 18

6.9 Faculty Menu – My Subjects 19

6.10 Faculty Menu – Subject Results 19

6.11 Faculty Menu – Pass Percentage 20

6.12 Student Menu – Full Result 20

6.13 Student Menu – Subject Performance 21

6.14 Student Menu – My Courses 21

6.15 Student Menu – Grade Summary 22

6. CONCLUSION 23

7. FUTURE ENHANCEMENTS 24

8. REFERENCES 25

APPENDIX (Sample Code) 26

APPENDIX (SQL Queries) 28

ABSTRACT

The Student Result Management System is a database-driven application developed to efficiently manage and process student academic records. The system is designed to reduce manual work involved in maintaining student results and to ensure accuracy, security, and easy accessibility of data. It provides a centralized platform where administrators, faculty members, and students can interact based on their roles.

The administrator is responsible for managing core data such as student details, subjects, course registrations, and faculty assignments. Faculty members are authorized to enter and update marks only for the subjects assigned to them, ensuring proper access control and data integrity. Students can log in to the system to view their results, subject-wise performance, course details, and overall grade summary.

The system is developed using Java for application logic and Oracle Database for backend data storage, with JDBC used for database connectivity. It supports various functionalities such as marks entry, result generation, grade calculation, pass/fail analysis, and performance reporting.

Overall, the system improves efficiency, minimizes errors, and provides a user-friendly interface for managing academic results. It can be further enhanced with graphical interfaces, web integration, and additional analytics features to meet real-world institutional requirements.

LIST OF FIGURES

Figure no.	Figure Title	Page no.
1	Login Screen of Student Result Management System	16
2	Admin Menu Screen	16
3	Student Registration Screen	17
4	Subject Management Screen	17
5	Course Registration Screen	18
6	Faculty Assignment Screen	18
7	Marks Entry Screen	19
8	Student Result View Screen	20
9	Subject-wise Performance Screen	21
10	Grade Summary Screen	22

LIST OF TABLES

Table no.	Table Title	Page no.
1	Student	9
2	Subject	10
3	Marks	10
4	Course_Registration	11
5	Course_Registration Relational Database	11

ACRONYMS

- **SRMS : Student Result Management System**
- **JDBC : Java Database Connectivity**
- **DBMS : Database Management System**
- **UI : User Interface**
- **SQL : Structured Query Language**
- **OOP : Object Oriented Programming**
- **SGPA : Semester Grade Point Average**

1. INTRODUCTION

1.1 Brief Information about the project

The Student Result Management System is a Java-based desktop application. It automates the process of managing student academic records efficiently.Users such as admin, faculty, and students can interact with the system based on their roles.The system allows adding student details, assigning subjects, entering marks, and viewing results.It ensures accurate calculation of grades, SGPA, and overall performance.All student, subject, and result data are stored and managed in a database.Developed using Core Java, it provides a text-based interface for easy interaction through the console.JDBC is used for database connectivity with Oracle Database as backend.The system follows a modular design for easy maintenance and scalability.It reduces manual work, minimizes errors, and improves efficiency in result management.This project demonstrates the practical implementation of Java and database management

1.2 Motivation and Contribution of Project

In many educational institutions, student result processing is still handled manually or using basic tools like spreadsheets. This often leads to errors in calculations, difficulty in maintaining records, lack of proper data security, and delays in result processing. These challenges motivated the development of the Student Result Management System.

The key contributions of this project include:

* Providing a digital solution for managing student academic records
* Reducing manual effort and paperwork in result processing
* Ensuring accurate calculation of marks, grades, and SGPA
* Enabling quick access to student performance and subject-wise results
* Improving data security through role-based login (admin, faculty, student)
* Simplifying course registration, marks entry, and result viewing

This system enhances efficiency, accuracy, and reliability in managing student results and provides a structured solution for academic institutions.

1.3 Objectives of the Project

The main objectives of the Student Result Management System are:

* To develop a user-friendly application for managing student academic record
* To store student, subject, faculty, and marks information in a structured database
* To provide functionalities for adding students, assigning subjects, and entering marks
* To enable quick calculation of grades, SGPA, and overall performance
* To allow students to view their results and subject-wise performance easily
* To display student records and results in an organized and readable format

1.4 Scope of the Project

The Student Result Management System can be used by educational institutions such as schools and colleges to manage student results efficiently. It supports operations such as student registration, course registration, faculty assignment, marks entry, and result generation. The system allows students to view their results, subject performance, and grade summaries through a simple interface.The system provides role-based access for admin, faculty, and students, ensuring proper control and data security. It offers a centralized database for storing and managing academic records, improving accuracy and reducing manual errors. The application is designed to be simple, efficient, and scalable, making it suitable for real-world academic environments.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Student Result Management System must perform to manage student records and results efficiently. The functional requirements of the Student Result Management System include:

• Add Student Details

The system should allow the admin to add new student records including student ID, name, branch, semester, and login details. Proper validation should ensure accurate data entry.

• Manage Subjects

The system should provide functionality to add and manage subject details such as subject ID, subject name, and branch association.

• Course Registration

The system should allow registration of students for different subjects. It should ensure that valid student and subject details are used.

• Assign Faculty

The system should allow the admin to assign faculty members to specific subjects for managing academic activities.

• Enter and Update Marks

The system should allow faculty to enter and update student marks for assigned subjects. It should ensure that only authorized faculty can modify marks.

• View Student Results

The system should allow users to view student results, including average marks, grades, and performance summaries.

• Subject-wise Performance Analysis

The system should display student performance in each subject, including marks, grades, and pass/fail status.

• Generate Reports

The system should generate reports such as topper list, course-wise registration count, and pass/fail statistics.

• Search Student Records

The system should support searching for student details and results using criteria quickly.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the Student Result Management System must satisfy.

• Performance

The system should respond quickly to operations such as adding student records, entering marks, calculating results, and retrieving data without delays.

• Security

Only authorized users (admin, faculty, and students) should access the system through proper login authentication. Role-based access control should ensure that users can only perform permitted operations.

• Usability

The system should have a simple and user-friendly interface, allowing users to easily perform tasks such as managing student records, entering marks, and viewing results without difficulty.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the Student Result Management System.

2.3.1 Minimum Hardware Requirements

* Processor: Intel i3 or above
* RAM: 4 GB (minimum)
* Hard Disk: 20 GB free space
* Monitor: Standard display
* Keyboard and Mouse

2.3.2 Software Requirements

* Operating System: Windows 7/10/11
* Programming Language: Java
* IDE: VS Code
* Database: Oracle Database
* Connectivity: JDBC

3. TECHNOLOGY DESCRIPTION

3.1 Programming language

3.1.1 Introduction to java

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It is widely used for developing desktop, web, and enterprise applications due to its platform independence and robustness.In the Student Result Management System, Java is used to implement the core functionality of the application. It handles operations such as managing student records, assigning subjects, entering marks, calculating grades, and generating results. Java provides features like object-oriented programming, exception handling, and strong memory management, which help in building reliable and efficient applications.One of the main advantages of Java is that it is platform-independent, meaning the program can run on any system that has a Java Virtual Machine (JVM). This makes it suitable for developing portable applications like the Student Result Management System.

3.2 Framework / Libraries

3.2.1 Console-Based Interface

The Student Result Management System uses a text-based interface in the console for user interaction.Users can perform operations by entering commands or selecting menu options displayed in the console, such as:

* Logging in (Admin / Faculty / Student)
* Adding Student Details
* Assigning Subjects and Courses
* Entering and Updating Marks
* Viewing Results and Performance
* Generating Reports and Statistics

The console-based interface is simple, easy to use, and allows efficient interaction without the need.

3.3 Database Technology

The Student Result Management System uses a relational database for storing and

managing student, subject, branch, faculty, and marks information in a structured manner. The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and consistency in academic records.In this project, Oracle Database is used as the backend, and JDBC (Java Database Connectivity) is used to connect the Java application to the database, enabling operations such as registering new students, assigning subjects to faculty, entering examination marks, and fetching performance results.

3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) in Java used to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database.

In the Student Result Management System, JDBC is used to connect the Java application with the Oracle Database. It performs operations such as:

• Establishing a connection to the database

• Executing SQL queries (INSERT, UPDATE, DELETE, SELECT) for managing student records, marks entry, and result processing

• Retrieving data about students, subjects, faculty assignments, and results

• Closing the database connection

JDBC acts as a bridge between the application and the database, ensuring smooth communication, maintaining data consistency, and enabling efficient data handling for student result management.

3.3.2 Oracle Database

Oracle Database is a powerful relational database management system used for storing and managing structured data. It provides high performance, reliability, and security for handling large volumes of data.In the Student Result Management System, Oracle Database is used to store student, subject, faculty, and marks details. It supports SQL for data manipulation and ensures data integrity through constraints such as primary keys, foreign keys, and proper data types. The database enables organized management of academic records and accurate result processing.

3.4 Tools & IDEs

Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Student Result Management System. It is a lightweight code editor that supports Java programming through extensions.VS Code provides features such as syntax highlighting, code completion, and debugging support, which help improve coding efficiency. Its simple and user-friendly interface makes it suitable for developing, testing, and maintaining the Student Result Management System.

4. DATABASE DESIGN

4.1 Database Tables

The Student Result Management System uses a relational database to store information in a structured manner. The database consists of multiple tables such as Student, Subject, Marks, and Course Registration.

* The Student table stores details of students including student ID, name, branch, semester, and username.

* The Subject table stores subject information including subject ID, subject name, and branch ID.

* The Marks table stores student marks for each subject and is linked with student and subject tables.

* The Course Registration table stores the mapping of students to subjects.

This separation of data helps in reducing redundancy and ensures organized management of student records and results.

4.2 Table Structure

Student Table

Field Name	Data Type	Description	Constraint
Student_id	NUMBER	Student ID	Primary Key
Student_name	VARCHAR2(100)	Name of the Student	Not Null
Branch_id	NUMBER	Branch ID	Not Null
Semester	NUMBER	Semester	Not Null
Username	VARCHAR2(50)	Login username	Unique

Table 1: Student Relational Database Table

Subject Table

Field Name	Data Type	Description	Constraint
Subject_id	NUMBER	Subject_ID	Primary Key
Subject_name	VARCHAR2(100)	Subject Name	Not Null
Branch_id	NUMBER	Branch ID	Not NullTable

Table 2: Subject Relational Database Table

Marks Table

Field Name	Data Type	Description	Constraint
Student_id	NUMBER	Student ID	Foreign Key
Subject_id	NUMBER	Subject ID	Foreign Key
Marks	NUMBER	Marks Obtained	-

Table 3: Marks Relational Database Table

Course Registration Table

Field Name	Data Type	Description	Constraint
Student_id	NUMBER	Student ID	Foreign Key
Subject_id	NUMBER	Subject ID	Foreign Key

Table 4: Course Registration Relational Database Table

Field Name	Data Type	Description	Constraint
Reg_id	NUMBER	Registration ID	Primary Key
Student_ID	NUMBER	Student ID	Foreign Key
Subject_code	VARCHAR2(10)	Subject Code	Foreign Key
Registration_date	DATE	Date of enrollment	Not Null
Academic_year	VARCHAR2(10)	Current Academic Year	-

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

*** Primary Keys:**

o student_id in Student table

o subject_id in Subject table

*** Foreign Keys:**

o student_id and subject_id in Marks table

o student_id and subject_id in Course Registration table

*** NOT NULL Constraint:**

o Important fields such as student_id, student_name, subject_name, and branch_id cannot be left empty

*** Data Type Constraints:**

o Each field accepts only valid data types to ensure correctness (e.g., NUMBER for IDs, VARCHAR2 for names)

These constraints help maintain data integrity, avoid duplication, and ensure that only valid and consistent data is stored in the database.

4.4 Table Relationships

The tables in the Student Result Management System are interconnected through common fields.

* Each student can register for multiple subjects.

* Each subject can have multiple students.

* The Marks table links students and subjects by storing marks obtained by each student in each subject.

This represents a many-to-many relationship between students and subjects, resolved using the Course Registration and Marks tables

5. SYSTEM DESIGN

5.1 System Architecture

The **STUDENT RESULT MANAGEMENT SYSTEM** follows a **three-layer architecture**:

1.Presentation Layer (Console Interface)

- User interacts with the system through console menus and prompts
- Operations include managing students, entering marks, and viewing results

2. Application Layer (Business Logic)

- Processes user inputs from the console
- Performs operations such as adding students, assigning subjects, calculating results, and generating reports

3.Data Layer (Database)

- Stores student, subject, marks, and course registration data
- Managed using Oracle Database

5.2 Interface Implementation

The system is divided into the following modules:

5.2.1. Student Management Module

- Stores student details such as ID, Name, Branch, and Semester
- Allows administrators to add, update, and manage student records

5.2.2. Marks and Result Module

- Handles marks entry and updates for various subjects
- Calculates total marks and determines pass/fail status for students

5.2.3. Subject Management Module

- Allows registration of different subjects and course codes
- Maintains the mapping between students and their respective subjects

5.2.4. Database Connectivity Module

- Connects the Java application with Oracle DB using JDBC

- Executes SQL queries for storing and retrieving academic data safely

5.2.5. Console Interface Module

- Provides a menu-driven interface in the console for easy navigation

- Handles user interactions such as viewing marks and generating result summaries

5.2.6. User Authentication Module

- Manages secure access to the system through a login interface for different roles

- Validates credentials from the **Login** table to distinguish between Admin and

5.2.7. Faculty Assignment Module

- Maps specific faculty members to the subjects they are responsible for teaching

- Ensures that marks can only be entered by the authorized faculty for a given subject

5.2.8. Branch and Department Module

- Maintains the list of academic branches such as Computer Science, IT, or Electronics

6.SYSTEM IMPLEMENTATION

6.1 Login Screen.

A console-based login system showing role selection (Admin, Faculty, Student) with a successful admin login.

```
--- LOGIN ---
1. Admin
2. Faculty
3. Student
Select Role: 1
Username: admin
Password: admin123
Login Successful as admin
```

6.2 Admin Menu and Add Student.

This module allows the admin to access the system menu and add new student details into the database.

```
1. User Login
2. Register
3. Admin Login
4. Exit
Choose: 2
Enter username: alice
Enter password: 123
Registration Successful!

1. User Login
2. Register
3. Admin Login
4. Exit
Choose: |
```

```
--- ADMIN MENU ---
1.Add Student
2.Add Subject
3.Register Course
4.Assign Faculty
5.View Registrations
6.View Results
7.Topper List
8.Course Count
9.Pass/Fail Stats
10.Performance Table
11.Logout
Choose option:1
Student ID: 112
Name: Sailesh
Branch ID: 8
Semester: 4
Username: s112
Student Added!
```

6.3 Register Course

A console-based admin menu displaying multiple management options, with a student successfully registered for a course.

```
--- ADMIN MENU ---
1.Add Student
2.Add Subject
3.Register Course
4.Assign Faculty
5.View Registrations
6.View Results
7.Topper List
8.Course Count
9.Pass/Fail Stats
10.Performance Table
11.Logout
Choose option:3
Student ID: 111
Subject ID: 12
Registered!
```

6.4 View Results

A console-based admin menu displaying student results with marks and corresponding grades.

```
--- ADMIN MENU ---
1.Add Student
2.Add Subject
3.Register Course
4.Assign Faculty
5.View Registrations
6.View Results
7.Topper List
8.Course Count
9.Pass/Fail Stats
10.Performance Table
11.Logout
Choose option:6
Kiran    60.0    C
Anil     40.0    D
Sai      30.0    F
Varun    72.0    B
Nikhil   35.0    F
Rahul    90.0    A
Teja     60.0    C
Ravi     71.5    B
```

6.5 Performance table

This image shows an admin menu of a student management system displaying performance table with students' marks and grades.

```
--- ADMIN MENU ---
1.Add Student
2.Add Subject
3.Register Course
4.Assign Faculty
5.View Registrations
6.View Results
7.Topper List
8.Course Count
9.Pass/Fail Stats
10.Performance Table
11.Logout
Choose option:10
Kiran    60.0    C
Anil     40.0    D
Sai      30.0    F
Varun    72.0    B
Nikhil   35.0    F
Rahul    90.0    A
Teja     60.0    C
Ravi     71.5    B
```

6.6 Logout

The system shows an academic admin menu where the user logged out successfully

```
--- ADMIN MENU ---
1.Add Student
2.Add Subject
3.Register Course
4.Assign Faculty
5.View Registrations
6.View Results
7.Topper List
8.Course Count
9.Pass/Fail Stats
10.Performance Table
11.Logout
Choose option:11

Logged out successfully!
```

6.7 Faculty Menu

The system displays a faculty menu with options for managing subjects, marks, and results.

```
--- FACULTY MENU ---
1.Enter Marks
2.My Subjects
3.Subject Results
4.Pass Percentage
5.Logout
Choose option:
```

6.8 Faculty Menu-Enter Marks

The faculty menu interface shows a marks entry example where a student's score is saved successfully.

```
--- FACULTY MENU ---
1.Enter Marks
2.My Subjects
3.Subject Results
4.Pass Percentage
5.Logout
Choose option:1
Subject ID: 12
Student ID: 111
Marks: 100
Marks Saved!
```

6.9 Faculty Menu-My Subjects

The faculty menu interface shows a marks entry example where a student's score is saved successfully.

```
--- FACULTY MENU ---
1.Enter Marks
2.My Subjects
3.Subject Results
4.Pass Percentage
5.Logout
Choose option:2
Machines
Thermodynamics
Python
AI Basics
Computer Vision
```

6.10 Faculty Menu - Subject Results

The faculty menu shows subject results where Geeth scored 100 marks and passed.

```
--- FACULTY MENU ---
1.Enter Marks
2.My Subjects
3.Subject Results
4.Pass Percentage
5.Logout
Choose option:3
Subject ID: 12
Geeth   100     PASS
```

6.11 Faculty Menu - Pass Percentage

The faculty menu shows a subject's pass percentage as 100%.

```
--- FACULTY MENU ---
1.Enter Marks
2.My Subjects
3.Subject Results
4.Pass Percentage
5.Logout
Choose option:4
Subject ID: 12
Pass %: 100.0
```

6.12 Student Menu- Full Result

The student dashboard shows Ravi's result in CSE, semester 3, with Networks graded as AB.

```
--- STUDENT DASHBOARD ---
1.Full Result
2.Subject Performance
3.My Courses
4.Grade Summary
5.Logout
Choose option:1

===== RESULT =====
Name: Ravi
Branch: CSE
Semester: 3
Networks : AB
```

6.13 Student Menu - Subject performance

The student dashboard shows Ravi's subject performance where Networks is graded AB and marked as FAIL.

```
--- STUDENT DASHBOARD ---
1.Full Result
2.Subject Performance
3.My Courses
4.Grade Summary
5.Logout
Choose option:2
Networks        AB      FAIL
```

6.14 Student Menu - My Courses

The student dashboard shows the course list, displaying "Networks" as Ravi's enrolled subject.

```
--- STUDENT DASHBOARD ---
1.Full Result
2.Subject Performance
3.My Courses
4.Grade Summary
5.Logout
Choose option:3
Networks
```

6.15 Student Menu - Grade Summary

The student dashboard grade summary shows 0 passes, 1 fail, and 1 absent.

```
--- STUDENT DASHBOARD ---
1.Full Result
2.Subject Performance
3.My Courses
4.Grade Summary
5.Logout
Choose option:4
Pass: 0 Fail: 1 Absent: 1
```

7.CONCLUSION

The Student Result Management System has been successfully developed to manage academic records and student evaluations efficiently and accurately. The system replaces traditional manual grading and record-keeping with a computerized solution, reducing human errors, saving administrative time, and improving data integrity.

The application provides essential functionalities such as student and subject registration, faculty assignment, marks entry, and automated result generation through a role-based console interface. By utilizing technologies like Java, JDBC, and Oracle Database, the system ensures robust performance, secure data handling, and real-time access to academic performance metrics.

Overall, the system improves the efficiency of managing institutional data, prevents unauthorized access through secure logins, and streamlines the workload for both administrators and faculty. It also provides a scalable foundation that can be enhanced in the future by adding features such as automated SGPA/CGPA calculation, PDF report generation, and an online web-based portal for parents.

In conclusion, the Student Result Management System meets the project objectives and serves as an effective tool for managing educational records in a reliable, organized, and transparent manner.

8. FUTURE ENHANCEMENTS

The Student Result Management System developed in this project provides basic functionalities for student registration, marks entry, and result viewing. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.
Some possible future enhancements include:

.

Web-Based Portal and Mobile Access:

Converting the console-based system into a full-stack web or mobile application, allowing students and faculty to access results and academic records remotely.·

Automated SGPA and CGPA Calculation:

Implementing advanced algorithms to automatically calculate Semester Grade Point Average (SGPA) and Cumulative Grade Point Average (CGPA) based on credit hours.

.

Automated Notifications and Alerts:

Integrating email or SMS services to send instant notifications to students when results are published or when attendance falls below a certain threshold.

.

Report Generation and Exporting:

Adding features to generate official transcripts, mark sheets, and performance charts in PDF or Excel formats for institutional documentation.

.

Data Analytics and Visualizations:

Generating graphical reports for class-wise pass percentages, topper comparisons, and subject-wise difficulty analysis for better academic management.These enhancements can make the system more scalable, user-friendly, and suitable for real-time use in educational institutions.

9.REFERENCES

The following references were used during the development of the Employee Management System:

The following references were used during the development of the Student Result Management System:

1. Herbert Schildt, Java: The Complete Reference, Oracle Press, 11th Edition.

2. E. Balagurusamy, Programming with Java, McGraw Hill Education.

3. Korth, Silberschatz, Sudarshan, Database System Concepts, McGraw Hill.

4. Oracle Database Documentation, https://docs.oracle.com/en/database/

5. JDBC API Documentation (Oracle), https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

6. Oracle SQL Tutorial – GeeksforGeeks, https://www.geeksforgeeks.org/sql-tutorial/

7. Java Database Connectivity Tutorial – TutorialsPoint, https://www.tutorialspoint.com/jdbc/

8. W3Schools SQL Tutorial, https://www.w3schools.com/sql/

APPENDIX

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the console-based Java application and the Oracle database using JDBC. It allows the application to perform operations such as inserting student records, entering marks, and retrieving academic results directly through the console interface.

Database Connection (DBConnection.java)

This class establishes a connection between the console-based Java application and the Oracle database using JDBC. It provides a reusable method getConnection() that returns a Connection object, allowing other modules to execute SQL queries for managing student data and examination results.

```
import java.sql.*;
public class DBConnection {
   public static Connection getConnection() {
      Connection con = null;
      try {
         // Load Oracle JDBC Driver
         Class.forName("oracle.jdbc.driver.OracleDriver");
           // Establish connection to the Oracle database
         con = DriverManager.getConnection(
  "jdbc:oracle:thin:@localhost:1521:xe", "system", "password");
      } catch (Exception e) {
         System.out.println(e);
```

```
        }
        return con;
    }
}
```

Insert Student Marks (InsertMarks.java)

This console-based Java program adds academic marks for a student into the database using JDBC.

The user provides marks details which the program then processes using an SQL INSERT query to store the information in the Marks table.

```
import java.sql.*;


public class InsertMarks {
    public static void main(String[] args) {
        try {
                    // Get database connection
            Connection con = DBConnection.getConnection();
            // SQL query to insert marks details
            String query = "INSERT INTO marks VALUES(?,?,?,?,?)";
            PreparedStatement pst = con.prepareStatement(query);
```

21

```
           // Set Marks data (can be read from Scanner)
            pst.setInt(1, 501);          // Result ID
            pst.setInt(2, 1001);         // Student ID
            pst.setString(3, "CS101");    // Subject Code
```

```
            pst.setInt(4, 85);          // Marks Obtained
            pst.setInt(5, 1);           // Semester
// Execute the query
            int i = pst.executeUpdate();
            if (i > 0)
                System.out.println("Marks Added Successfully");
// Close the connection
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

A.2 SQL Queries

1.Create Login Table

```
CREATE TABLE login (
    username VARCHAR2(50) PRIMARY KEY,
    password VARCHAR2(50),
    role VARCHAR2(20)
);
```

2.Create branch table

```
CREATE TABLE branch (
    branch_id NUMBER PRIMARY KEY,
    branch_name VARCHAR2(50)
```

```
);
```

3.Create student table

```
CREATE TABLE student (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(50),
    branch_id NUMBER,
    semester NUMBER,
    username VARCHAR2(50) UNIQUE,
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);
```

4.Create subject table

```
CREATE TABLE subject (
    subject_id NUMBER PRIMARY KEY,
    subject_name VARCHAR2(50),
    branch_id NUMBER,
    max_marks NUMBER,
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);
```

5. Create course_registration table

```
CREATE TABLE course_registration (
    student_id NUMBER,
    subject_id NUMBER,
```

```
    PRIMARY KEY (student_id, subject_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
);
```

6.Create marks table

```
CREATE TABLE marks (
    student_id NUMBER,
    subject_id NUMBER,
    marks NUMBER,
    PRIMARY KEY (student_id, subject_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
);
```

7.Create faculty_subject table

```
CREATE TABLE faculty_subject (
    faculty_username VARCHAR2(50),
    subject_id NUMBER,
    PRIMARY KEY (faculty_username, subject_id),
    FOREIGN KEY (faculty_username) REFERENCES login(username),
    FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
;
```